Fundtir Token Smart Contract

Overview

The **FundtirToken** contract is the official ERC20 token of the Fundtir ecosystem.
It is a **standard and secure fungible token** with burn and permit extensions, designed to support efficient transfers, decentralized finance (DeFi) integration, and secure supply management.

The contract follows **OpenZeppelin's audited ERC20 standards**, ensuring industry-grade security and compatibility with wallets, exchanges, and DeFi protocols.

Key Features

1. **ERC20 Standard Compliance**
    - Fully compatible with all Ethereum-based wallets, exchanges, and DeFi platforms.
    - Provides the standard functions: transfer, approve, transferFrom, allowance, balanceOf, and totalSupply.
2. **Fixed Supply**
    - **Total Supply:** 700,000,000 FNDR.
    - Entire supply is minted once at deployment and assigned to the **admin wallet**.
    - No additional tokens can ever be minted, ensuring scarcity and predictability.
3. **Burnable (ERC20Burnable)**
    - Holders can **irreversibly destroy (burn)** their tokens, reducing total supply.
    - Useful for deflationary mechanisms, token buybacks, or community-driven burns.
4. **Gasless Approvals (ERC20Permit, EIP-2612)**
    - Enables token approvals via **signatures** instead of on-chain transactions.

    - Users can authorize spending without paying gas fees, improving UX for DeFi and dApps.

5. **Ownership Management (Ownable2Step)**
    - Two-step ownership transfer process ensures security:
        - transferOwnership(newOwner) → Initiates transfer.
        - acceptOwnership() → New owner confirms.
    - Prevents accidental ownership loss and supports multi-admin governance models.

Deployment Details

- **Token Name:** Fundtir
- **Symbol:** FNDR
- **Decimals:** 18
- **Initial Supply:** 700,000,000 * 10^18 FNDR
- **Admin Wallet:** Entire supply minted to _adminWallet address at deployment.

Contract Functions

Core ERC20

- transfer(address to, uint256 amount) → Send tokens to another wallet.
- approve(address spender, uint256 amount) → Approve spender to use tokens.
- transferFrom(address from, address to, uint256 amount) → Move tokens via allowance.
- balanceOf(address account) → Returns account balance.
- totalSupply() → Returns total circulating supply.

Burning

- burn(uint256 amount) → Destroy caller's tokens.
- burnFrom(address account, uint256 amount) → Burn tokens from an approved account.

Permit (EIP-2612)

- permit(owner, spender, value, deadline, v, r, s) → Approve spending with a signed message.
- Removes need for ETH gas fees on approval.

Ownership

- transferOwnership(address newOwner) → Initiate ownership transfer.
- acceptOwnership() → New owner accepts.

Security Considerations

- **OpenZeppelin Libraries:** Ensures best practices and reduces risks of implementation bugs.
- **Fixed Supply:** No minting functions included, preventing inflation.
- **Burn Feature:** Deflationary option, but only voluntary by token holders or approved spenders.
- **Two-Step Ownership:** Prevents admin loss from accidental ownership transfer.
- **EIP-2612 Support:** Enhances security by reducing need for multiple on-chain approvals.

Fundtir PreSale Smart Contract

Overview

The **FundtirPreSale** contract enables participants to purchase **Fundtir (FNDR)** tokens using **USDT** during the presale phase. It ensures secure token transfers, configurable parameters (token price, minimum purchase limit), and treasury management for fund collection.

Key Features

- **USDT Payments** – Purchases are made using the USDT stablecoin.
- **Configurable Price** – FNDR token price (in 18 decimals) can be updated by the admin.
- **Minimum Threshold** – Ensures users cannot buy below the defined USDT threshold.
- **Pausable Contract** – Emergency stop mechanism to halt presale operations.
- **Reentrancy Guard** – Prevents reentrancy attacks in purchase flows.
- **Treasury Wallet** – All USDT payments are directed to a treasury wallet.
- **Admin Controls** – Owner can update price, threshold, treasury wallet, and withdraw tokens.

Contract Details

- **Solidity Version:** ^0.8.24
- **License:** MIT
- **Inheritance:**
    - Ownable2Step – Ownership management with secure two-step transfers.
    - Pausable – Pause/unpause presale operations.
    - ReentrancyGuard – Security against reentrancy.
- **Libraries Used:**
    - SafeERC20 – Secure ERC20 transfers.

State Variables

Variable	Type	Description
tokensBought	mapping	Tracks total FNDR purchased by each address.
treasuryWallet	address	Destination address for USDT payments.
fundtir	IERC20	Fundtir (FNDR) token contract.
usdtToken	IERC20	USDT token contract used for payments.
tokenPrice	uint256	FNDR price in USDT (18 decimals).
minThresholdLimit	uint256	Minimum purchase limit in USDT (6 decimals).

Events

Event	Parameters	Description
TokensPurchased	buyer, amount	Emitted when tokens are purchased.
minThresholdUpdated	newMinThresholdLimit	Emitted when minimum threshold changes.
TokenPriceUpdated	oldPrice, newPrice	Emitted when FNDR price changes.
TokensWithdrawn	admin, amount	Emitted when tokens are withdrawn by admin.
TreasuryWalletUpdated	newWallet	Emitted when treasury wallet is updated.

Constructor

```
constructor(
    uint256 _tokenPrice,
    address _treasuryWallet,
    uint256 _minThresholdLimit,
    address _Fundtiren,
    address _usdttoken
) Ownable(_treasuryWallet)
```

Parameters:

- _tokenPrice – Price of FNDR token in USDT (18 decimals).
- _treasuryWallet – Address that collects USDT and becomes owner.

- _minThresholdLimit – Minimum USDT purchase (6 decimals).
- _Fundtiren – Address of FNDR token.
- _usdttoken – Address of USDT token.

Requirements:

- Valid addresses.
- _tokenPrice > 0.
- _minThresholdLimit > 0.

Functions

View Functions

```
previewFundtirForUSDT(uint256 usdtAmount) external view returns (uint256)
```

- Estimates how many FNDR tokens can be bought for a given USDT amount.

Formula:

```
(usdtAmount * 1e18 * 1e12) / tokenPrice
```

- Returns FNDR tokens in 18 decimals.

Purchase Functions

```
buyTokens(uint256 usdtAmount) external whenNotPaused nonReentrant
```

- Allows users to purchase FNDR with USDT.

Requirements:

- Contract must not be paused.
- usdtAmount > 0.

- usdtAmount >= minThresholdLimit.
- Contract must hold enough FNDR.
- Buyer must have sufficient USDT balance & allowance.

Process:

1. Calculate FNDR tokens based on tokenPrice.
2. Transfer usdtAmount from buyer → treasury wallet.
3. Transfer FNDR tokens from contract → buyer.
4. Update tokensBought mapping.
5. Emit TokensPurchased.

Admin Functions

updateTreasuryWallet(address _treasuryWallet)

- Updates treasury wallet.
- Must be non-zero and different from current wallet.

setTokenPrice(uint256 _newTokenPrice)

- Updates FNDR price in 18 decimals.

setMinThreshold(uint256 _minThresholdLimit)

- Updates minimum purchase threshold (6 decimals).

pause() & unpause()

- Pauses/unpauses the contract.

Withdrawal Functions

withdrawTokens(address tokenAddress, uint256 amount)

- Allows owner to withdraw any ERC20 tokens from contract.
- Transfers tokens to treasury wallet.
- Adjusts amount if greater than available balance.

Security Considerations

1. **Reentrancy Protection** – nonReentrant modifier prevents double-spending in buyTokens.
2. **Pausable** – Admin can stop presale in emergencies.
3. **Safe Transfers** – SafeERC20 ensures safe token operations.
4. **Admin Restrictions** – Only owner can modify price, threshold, treasury, or withdraw tokens.

FNDR Staking Smart Contract

Overview

The **FNDRStaking** contract allows users to stake **Fundtir (FNDR)** tokens and earn rewards in **USDT**. It combines fixed-term staking with APY-based interest, dividend distributions, and governance checkpointing for DAO voting power.

Key Features

- **Multiple staking plans** – Different APYs and lock durations.
- **USDT rewards** – Interest calculated in FNDR but paid in USDT.
- **Dividend distributions** – Admin can distribute USDT to long-term stakers via snapshot eligibility.
- **Governance checkpointing** – Tracks staked balances at specific blocks for DAO voting power.

- **Dynamic FNDR price** – Admin sets FNDR→USDT conversion rate.
- **Reentrancy protection** – Secure staking and reward claiming.
- **Admin controls** – Adjustable APY, durations, token management.

Staking Plans

Plan	Duration	APY (Basis Points)	APY (%)
1	90 days	897	8.97%
2	365 days	1435	14.35%
3	730 days	2152	21.52%
4	1460 days	2869	28.69%

Contract Details

- **Solidity Version:** ^0.8.24
- **License:** MIT
- **Inheritance:**
  - Ownable2Step – Secure ownership transfers (recommended for multisig).
  - ReentrancyGuard – Prevents reentrancy attacks.
- **Libraries Used:**
  - SafeERC20 – Secure ERC20 transfers.

State Variables

Token Contracts

- fundtirToken – FNDR staking token (immutable).
- usdtToken – USDT reward token.

Pricing

- fndrPriceInUSDT – Price of FNDR in USDT (6 decimals).

Staking

- totalStakedAmount – Total FNDR staked across all users.
- stakes[user] – Array of stakes for each user.
- _currentStaked[user] – Active staked FNDR.
- stakeHolders – List of all addresses that have ever staked.

APY & Durations

- PLAN_X_APY – APY per plan (basis points).
- PLAN_X_DAYS – Lock duration per plan (seconds).

Governance

- _checkpoints[user] – Staking checkpoints for DAO voting.

Dividends

- distributionCounter – Sequential ID for distributions.
- distributions[id] – Dividend details.
- hasClaimed[id][user] – Tracks whether a user has claimed from a distribution.

Events

- Staked(user, amount, duration, apy) – User staked FNDR.
- Unstaked(user, amount, usdtInterest) – User unstaked and received rewards.
- CheckpointUpdated(user, block, stake) – DAO checkpoint update.
- FNDRPriceUpdated(newPrice) – FNDR price updated.
- DistributionStarted(id, snapshotBlock, totalAmount, eligibleTotal) – Dividend round started.
- DividendClaimed(id, user, amount) – User claimed dividends.
- AdminDeposit(admin, token, amount) – Admin deposited tokens.
- AdminWithdraw(admin, token, amount) – Admin withdrew tokens.
- APYUpdated(p1, p2, p3, p4) – Staking plan APYs updated.
- PlanDaysUpdated(d1, d2, d3, d4) – Staking durations updated.

Core Functions

Staking

stake(uint256 amount, uint256 plan)

- Locks FNDR tokens for selected plan.
- Creates a stake record.
- Updates DAO checkpoints.

unstake(uint256 stakeIndex)

- Unlocks FNDR + interest (in USDT).
- Requires plan duration completion.
- Updates user's checkpoints.

Reward Calculation

calculateInterest(uint256 amount, uint256 apy, uint256 duration)

Linear interest formula:

interest = (amount * apy * duration) / (365 days * 10000)

- Returns interest in FNDR (18 decimals).

convertFNDRToUSDT(uint256 fndrAmount)

- Converts FNDR → USDT using fndrPriceInUSDT.

previewUSDTInterest(uint256 fndrAmount, uint256 plan)

- Returns projected USDT rewards for given stake.

Governance (DAO Voting)

_writeCheckpoint(user, newStake)

- Records stake changes for voting power.

stakedAtBlock(user, blockNumber)

- Returns user's stake at past block (binary search).

Dividend Distributions

startDistribution(uint256 totalAmount)

- Admin starts a USDT dividend pool.
- Snapshot taken of eligible stakers.
- Eligible: Must stake ≥60 days.

claimFromDistribution(uint256 distributionId)

- Users claim their USDT share.

- Share = (userStake / eligibleTotal) * totalAmount.

Admin Functions

- updateFNDRPrice(uint256 newPrice) – Updates FNDR price (6 decimals).
- adminDeposit(address token, uint256 amount) – Admin deposits tokens (e.g., USDT for dividends).
- adminWithdraw(address token, uint256 amount, address to) – Admin withdraws tokens.
    - Restriction: Cannot withdraw staked FNDR.
- updateAPY(p1, p2, p3, p4) – Update plan APYs.
- updatePlanDays(d1, d2, d3, d4) – Update staking durations.

Security Considerations

1. **Reentrancy** – Protected by nonReentrant.
2. **SafeERC20** – Ensures safe token transfers.
3. **Admin Controls** – Restricted to onlyOwner.
4. **DAO Governance** – Voting power is snapshot-based, preventing manipulation.
5. **Dividend Eligibility** – Ensures only long-term stakers benefit.

Fundtir Vesting Smart Contract

1. Overview

The **FundtirVesting** contract is a comprehensive token vesting solution for Fundtir tokens, designed to manage multiple customizable vesting schedules for users.

It supports **linear vesting**, **instant unlocks**, **cliff periods**, and **custom release frequencies** while offering **role-based access control** for schedule management.

The system ensures fairness, transparency, and flexibility in distributing Fundtir tokens over time to investors, team members, advisors, and other stakeholders.

2. Key Features

- **Multiple Vesting Schedules per User**
  Each participant can hold multiple schedules with independent parameters.

- **Configurable Parameters**

  - **Instant Unlock (%):** Fraction of tokens available immediately.

  - **Cliff Period:** Delay before vesting starts.

  - **Vesting Period:** Total duration of vesting.

  - **Frequency:** Interval between releases.

- **Linear Vesting with Precision**
  Tokens vest gradually per defined frequency until completion.

- **Role-Based Access Control**

  - **DEFAULT_ADMIN_ROLE:** Full control, emergency withdrawals.

  - **ACCOUNT_MANAGER_ROLE:** Can create vesting schedules.

  - **Participants:** Can claim vested tokens.

- **Claim Flexibility**

  - Users can claim per schedule (claimTokens).

  - Users can claim across all schedules at once (claimAllTokens).

  - Admins can recover mistakenly sent tokens.

3. Contract Architecture

3.1 Roles

- **Admin (MultiSig)**

  - Owns the contract.

    - Can withdraw tokens in emergencies.
    - Grants/revokes roles.
- **Account Manager**
    - Can create vesting schedules for users.
    - Must fund the contract with vested tokens during creation.
- **Participant (User)**
    - Recipient of vesting schedules.
    - Allowed to claim vested tokens as per schedule.

3.2 Data Structures

VestingSchedule Struct

- scheduleId → Unique identifier.
- totalAmount → Total tokens allocated.
- instantUnlock → Percentage of tokens unlocked at creation.
- cliffPeriod → Duration before vesting starts.
- vestingPeriod → Total vesting duration.
- startTime → Creation timestamp.
- tokensReleased → Already claimed tokens.
- frequency → Interval of release.

3.3 Storage Mappings

- userVestingSchedules[user] → List of schedules for a user.
- isVestingParticipant[user] → Tracks if a user has schedules.
- nextScheduleId → Global counter for new schedules.

3.4 Events

- TokensReleased(user, scheduleId, amount) → Tokens successfully claimed.
- TokensWithdrawn(admin, amount) → Admin withdrawal executed.
- VestingScheduleCreated(user, scheduleId, params...) → New schedule added.

4. Workflow

4.1 Vesting Schedule Creation

1. Manager/Admin calls createVestingSchedule.
2. Contract validates parameters:
    - totalAmount > 0
    - instantUnlock ≤ 100
    - frequency > 0 and vestingPeriod ≥ frequency
3. Tokens are transferred from manager to contract.
4. If instantUnlock > 0, immediate tokens are released.
5. Event emitted for traceability.

4.2 Claiming Tokens

- **Per Schedule:**
    1. User calls claimTokens(scheduleId).
    2. Contract calculates releasable amount via getReleasableAmount.
    3. Updates tokensReleased.
    4. Transfers tokens to user.
- **All Schedules:**
    1. User calls claimAllTokens.
    2. Contract iterates through all schedules.

3. Accumulates releasable tokens.
4. Transfers total to user.

4.3 Releasable Token Calculation

- If **before cliff** → 0.
- If **after cliff** →
  - Compute total intervals = (vestingPeriod - cliffPeriod) / frequency.
  - Compute intervals passed.
  - Vest tokens linearly until all are released.
- Ensure final interval releases all remaining tokens.

4.4 Admin Withdrawals

- withdrawTokens(tokenAddress, amount) allows recovery of ERC20 tokens.
- Restricted to **contract owner (multiSig)**.
- Intended for **emergencies/mistaken transfers**.

5. Security Considerations

- **Reentrancy Protection:**
  All claim functions use nonReentrant.
- **Role Management:**
  Based on OpenZeppelin AccessControl.
- **Allowance Check:**
  Token transfer requires sufficient allowance.
- **Final Vesting Guarantee:**
  Ensures 100% of allocated tokens vest after the full period.
- **Emergency Recovery:**
  Owner can withdraw non-vested/mistaken tokens.

FNDR Governance Smart Contract

1. Overview

The **FNDRGovernance** contract implements a **decentralized governance system** for Fundtir token holders.

It enables community-driven decision-making through **proposal creation, voting, and execution** while integrating **staking** and **vesting contracts** to calculate voting eligibility and thresholds.

FNDRGovernance enforces a **"one wallet, one vote" model**, ensuring fair and democratic participation, with eligibility gated by **minimum stake/vested balance requirements**.

2. Key Features

- **Proposal System**
  - Anyone meeting the proposal threshold (default: 0.25% of total supply in combined staked + vested tokens) can create proposals.
  - Proposal lifecycle includes **creation** → **voting** → **execution**.
- **Voting Model**
  - **One wallet, one vote** system.
  - Voting options: **For / Against / Abstain**.
  - Minimum voting weight (e.g., 1000 FNDR) required to cast a valid vote.
- **Dynamic Quorum**
  - Admin-defined quorum determines the **minimum number of votes required** for execution.
- **Integration**
  - **Staking Contract**: Provides historic and current staked balances.
  - **Vesting Contract**: Includes unclaimed vested tokens in eligibility.
  - **FNDR Token Contract**: Used for supply-based thresholds.
- **Execution**

    - Manual execution of passed proposals by an **executor**.

- **Admin Configurability**
    - Adjust **proposal threshold, quorum, voting delay, voting period, and minimum voting weight**.

3. Contract Architecture

3.1 Roles

- **Executor / Owner**
    - Deploys and owns the contract.
    - Configures governance parameters.
    - Executes passed proposals.
- **Proposer (Eligible User)**
    - Must hold ≥ threshold (default 0.25% of total supply).
    - Can create governance proposals.
- **Voter (Participant)**
    - Must meet minimum voting weight (default: 1000 FNDR staked + vested).
    - Casts **one vote per wallet**.

3.2 Data Structures

Proposal Struct

- id → Unique identifier.
- proposer → Creator's address.
- title → Short descriptive title.
- description → Detailed description or IPFS/URL.
- startTime → Voting start timestamp.

- endTime → Voting end timestamp.
- executed → Whether executed.
- forVotes, againstVotes, abstainVotes → Voting tallies.
- totalVotes → Sum of all votes cast.
- snapshotBlock → Block used for historic balance lookup.

VoteReceipt Struct

- hasVoted → Prevents double voting.
- support → Vote type.
- weight → Voting weight (for eligibility check).
- votedAt → Timestamp of vote.

3.3 Storage Mappings

- proposals[proposalId] → Proposal details.
- receipts[proposalId][voter] → Vote receipts.

3.4 Events

- ProposalCreated → Emitted on proposal creation.
- VoteCast → Emitted when a vote is cast.
- ProposalExecuted → Emitted when a proposal is executed.
- VotingParamsChanged → Emitted when delay/period updated.
- QuorumVotesUpdated → Emitted when quorum changes.
- ProposalThresholdUpdated → Emitted when threshold changes.
- MinVotingWeightUpdated → Emitted when minimum voting weight changes.

4. Workflow

4.1 Proposal Creation

1. User calls propose(title, description).
2. Contract checks **eligibility threshold** = (totalSupply * proposalThresholdBps / 10000).
    - Eligibility includes **staked amount (snapshot)** + **unclaimed vested amount (current)**.
3. If valid:
    - Proposal stored in state.
    - Snapshot block recorded.
    - Event emitted.

4.2 Voting

1. Voting begins after votingDelay.
2. Users cast votes via castVote(proposalId, support).
    - Validates voting window.
    - Ensures user hasn't already voted.
    - Requires votingWeight ≥ minVotingWeight.
    - Each wallet → **1 vote only**.
3. Votes counted in **For / Against / Abstain** buckets.

4.3 Execution

1. After votingPeriod:
    - Check quorum: totalVotes ≥ requiredQuorumVotes.
    - Check majority: forVotes > againstVotes.
2. If valid:
    - Proposal marked executed = true.

    - ProposalExecuted emitted.
    - **Manual execution by executor** outside the contract.

5. Voting Power Calculation

- **Eligibility (Proposals & Votes)**
    - Based on combined:
        - StakedAmount (snapshot at proposal block)
        - UnclaimedVested = totalVested - totalReleased (current)
- **Weight in Voting**
    - Used only for eligibility.
    - Final vote count is **one per wallet**.

6. Security Considerations

- **Reentrancy Protection**: Voting & proposal execution are nonReentrant.
- **Snapshot Integrity**: Uses block-based snapshot for staking; vesting uses real-time data.
- **Double Voting Prevention**: Vote receipts enforce single vote per wallet.
- **Admin Parameters**: Owner (multisig) controls thresholds/quorum.
- **Manual Execution**: Ensures critical actions require multisig governance.